ARRAYS IN JAVA

- Java array is an object which contains elements of a similar data type. Additionally, The elements of an array are stored in a contiguous memory location. It is a data structure where we store similar elements. We can store only a **fixed set of** elements in a Java array.

Basic Declaration

Type[] arrayName;

This declares an array named arrayName that will store elements of type Type.

Example: **int[] myArray;** declares an array of integers.

Type arrayName[];

This is an alternative syntax that also declares an array. It's less preferred because it mixes the type with the array's name, which can be confusing in complex declarations.

Example: int myArray[]; also declares an array of integers.

Declaration with Allocation

To allocate memory for the array (i.e., specify the size of the array), use the new keyword:

```
Type[] arrayName = new Type[size];
```

This not only declares the array but also allocates space for size elements in the array.

Example: int[] myArray = new int[10]; declares and allocates space for an array of 10 integers.

Fixed ??????

In Java, when we say the length of an array is "fixed," it means that once an array is created with a specified number of elements, the size of the array cannot be changed. The capacity to store elements is determined at the time of array creation and remains constant throughout the lifetime of the array.

Initialization: When you create an array in Java, you must either specify the size of the array upfront or initialize it with a specific set of elements. For example,

`int[] myArray = new int[10];` creates an array that can hold 10 integers. Alternatively,

`int[] myArray = {1, 2, 3, 4, 5};` creates an array with 5 elements. In both cases, the size is fixed at creation.

No Resizing: After an array is created, you cannot add more elements to the array than it was initially declared to hold. If you attempt to store more elements than its declared capacity, you'll get an ArrayIndexOutOfBoundsException. For instance, attempting to access or assign a value to myArray[10] in an array declared with a size of 10 (indexes 0 through 9 are valid) would cause an error.

Advantages

Code Optimization: It makes the code optimized, we can retrieve or sort the data efficiently.

Random access: We can get any data located at an index position.

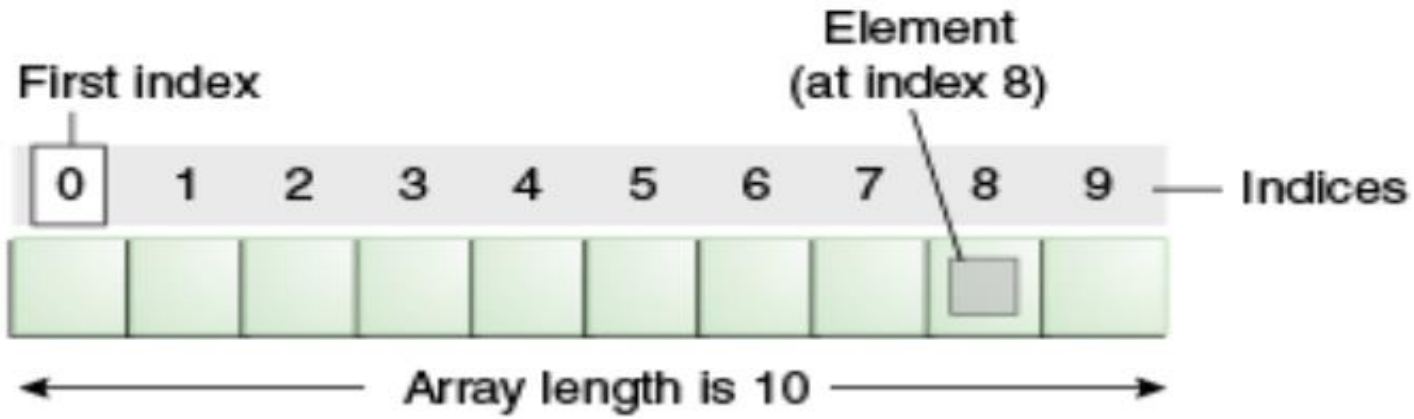
Disadvantages

Size Limit: We can store only the fixed size of elements in the array. It doesn't grow its size at runtime. To solve this problem, collection framework is used in Java which grows automatically.

Types of Array in java

There are two types of array.

- Single Dimensional Array
- Multidimensional Array



Declaration with Initialization

If you know the elements your array should contain at the time of its declaration, you can initialize it directly:

```
Type[] arrayName = {element0, element1, element2, ...};
```

This declares an array, allocates space for the given elements, and initializes it with the provided values.

Example: `int[] myArray = {1, 2, 3, 4, 5};` declares, allocates, and initializes an array of integers with 5 elements.

Multidimensional Array Declaration

For multidimensional arrays (arrays of arrays), the syntax extends naturally:

Type[][] arrayName;

Declares a two-dimensional array.

Example: int[][] matrix; declares a two-dimensional array (matrix) of integers.

Type[][] arrayName = new Type[rows][cols];

Declares and allocates a two-dimensional array with a specified number of rows and columns.

Example: int[][] matrix = new int[3][4]; allocates a 3x4 matrix.

Type[][] arrayName = {{row1col1, row1col2}, {row2col1, row2col2}, ...};

Declares, allocates, and initializes a two-dimensional array.

Example: int[][] matrix = {{1, 2}, {3, 4}}; creates a 2x2 matrix.

Initializing an Array in Separate Steps

You can declare an array and then allocate memory for it using the new keyword, and finally initialize its elements:

```
int[] array; // Declaration
array = new int[5]; // Allocation
array[0] = 1; // Initialization
array[1] = 2;
array[2] = 3;
array[3] = 4;
array[4] = 5;
```

Initializing a Multidimensional Array

For a two-dimensional array, you can use nested curly braces:

```
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
}; // A 3x3 matrix
```

This creates a 3x3 matrix where the elements are initialized to the specified values.

In the Java array, each memory location is associated with a number. The number is known as an array index. We can also initialize arrays in Java, using the index number. For example,

```
// declare an array
int[] age = new int[5];

// initialize array
age[0] = 12;
age[1] = 4;
age[2] = 5;
..
```

```
public class Main {
    public static void main(String[] args) {
        // Declare an array of integers
        int[] numbers;
        // Allocate memory for 5 integers
        numbers = new int[5];
        // Initialize elements
        numbers[0] = 10;
        numbers[1] = 20;
        numbers[2] = 30;
        numbers[3] = 40;
        numbers[4] = 50;
        // Access and print the third element
        System.out.println("The third number is: " + numbers[2]);
    } }
```

Example: Access Array Elements

```
class Main {
 public static void main(String[] args) {

   // create an array
   int age[] = {12, 4, 5, 2, 5};

   // access each array elements
   System.out.println("Accessing Elements of Array:");
   System.out.println("First Element: " + age[0]);
   System.out.println("Second Element: " + age[1]);
   System.out.println("Third Element: " + age[2]);
   System.out.println("Fourth Element: " + age[3]);
   System.out.println("Fifth Element: " + age[4]);
 }
}
```

Example: Access Array Elements

```
class Main {
 public static void main(String[] args) {

   // create an array
   int age[] = {12, 4, 5, 2, 5};

   // access each array elements
   System.out.println("Accessing Elements of Array:");
   System.out.println("First Element: " + age[0]);
   System.out.println("Second Element: " + age[1]);
   System.out.println("Third Element: " + age[2]);
   System.out.println("Fourth Element: " + age[3]);
   System.out.println("Fifth Element: " + age[4]);
 }
}
```

Output

```
Accessing Elements of Array:
First Element: 12
Second Element: 4
Third Element: 5
Fourth Element: 2
Fifth Element: 5
```

Java Operators

- Arithmetic Operators
- Auto-increment & decrement Operators
- Bitwise Operators
- Logical Operators
- Assignment Operators
- Comparison or Relational Operators
- Ternary Operator
- Shift Operators

Arithmetic Operators :

Arithmetic operators are used for basic mathematical operations.

- Addition (+): Adds two operands.
- Subtraction (-): Subtracts the second operand from the first.
- Multiplication (*): Multiplies two operands.
- Division (/): Divides the first operand by the second.
- Modulus (%): Returns the remainder of dividing the first operand by the second.

Examples:

```
System.out.println(10 + 5);  // 15
System.out.println(10 - 5);  // 5
System.out.println(10 * 5);  // 50
System.out.println(10 / 5);  // 2
System.out.println(10 % 5);  // 0
```

Relational Operators

Relational operators compare two operands and return a boolean value.

- Equal to (==): Checks if two operands are equal.
- Not equal to (!=): Checks if two operands are not equal.
- Greater than (>): Checks if the left operand is greater than the right.
- Less than (<): Checks if the left operand is less than the right.
- Greater than or equal to (>=): Checks if the left operand is greater than or equal to the right.
- Less than or equal to (<=): Checks if the left operand is less than or equal to the right.

Examples:

```
System.out.println(5 == 5);  // true
System.out.println(5 != 10); // true
System.out.println(5 > 3);   // true
System.out.println(5 < 10);  // true
System.out.println(5 >= 5);  // true
System.out.println(5 <= 2);  // false
```

Logical Operators

Logical operators are used to combine boolean expressions.

- AND (&&): True if both operands are true.
- OR (||): True if at least one of the operands is true.
- NOT (!): Inverts the value of the operand.

Examples:

```
System.out.println(true && false); // false
System.out.println(true || false); // true
System.out.println(!true);         // false
```

Assignment Operators

Assignment operators assign values to variables.

- **Assign (=):** Assigns the right operand to the left operand.
- **Add and assign (+=):** Adds the right operand to the left operand and assigns the result.
- **Subtract and assign (-=):** Subtracts the right operand from the left operand and assigns the result.
- **Multiply and assign (*=):** Multiplies the right operand with the left operand and assigns the result.
- **Divide and assign (/=):** Divides the left operand by the right operand and assigns the result.

Examples:

```
int a = 10;
a += 5;  // a = a + 5
a -= 3;  // a = a - 3
a *= 2;  // a = a * 2
a /= 4;  // a = a / 4
System.out.println(a); // 3
```

Unary Operators

Unary operators operate on a single operand.

- Increment (++): Increases the value of the operand by 1.
- Decrement (--): Decreases the value of the operand by 1.
- Positive (+): Indicates the positive value of the operand.
- Negative (-): Negates the value of the operand.
- Logical complement (!): Inverts the boolean value of the operand.

Examples:

```
int a = 10, b = -10;

boolean flag = false;

System.out.println(++a); // 11

System.out.println(--b); // -11

System.out.println(+a);  // 11

System.out.println(-a);  // -11
```

Bitwise Operators

Bitwise operators perform operations on individual bits of integer types.

AND (&): Bitwise AND.

OR (|): Bitwise OR.

XOR (^): Bitwise XOR (exclusive OR).

NOT (~): Bitwise NOT (invert bits).

Left shift (<<): Shifts bits to the left.

Right shift (>>): Shifts bits to the right.

Unsigned right shift (>>>): Shifts bits to the right, filling the left bits with zeros.

```
int a = 5; // 0101 in binary
int b = 3; // 0011 in binary
System.out.println(a & b); // 1 (0001)
System.out.println(a | b); // 7 (0111)
System.out.println(a ^ b); // 6 (0110)
```


Empty
CompareTo
Concat
EndsWith
String Methods
Replace
Equals

Java String

What is String in Java?

Generally, String is a sequence of characters. But in Java, string is an object that represents a sequence of characters. The java.lang.String class is used to create a string object.

How to create a string object?

There are two ways to create String object:

- **By string literal**
- **By new keyword**

1) String Literal

Java String literal is created by using double quotes. For Example:

```
String s="welcome";
```

2) By new keyword

```
String s=new String("Welcome");
//creates two objects and one reference variable
```

No.	Method	Description
1	char charAt(int index)	It returns char value for the particular index
2	int length()	It returns string length
3	static String format(String format, Object... args)	It returns a formatted string.
4	static String format(Locale l, String format, Object... args)	It returns formatted string with given locale.
5	String substring(int beginIndex)	It returns substring for given begin index.
6	String substring(int beginIndex, int endIndex)	It returns substring for given begin index and end index.
7	boolean contains(CharSequence s)	It returns true or false after matching the sequence of char value.

8	static String join(CharSequence delimiter, CharSequence... elements)	It returns a joined string.
9	static String join(CharSequence delimiter, Iterable<? extends CharSequence> elements)	It returns a joined string.
10	boolean equals(Object another)	It checks the equality of string with the given object.
11	boolean isEmpty()	It checks if string is empty.
12	String concat(String str)	It concatenates the specified string.
13	String replace(char old, char new)	It replaces all occurrences of the specified char value.
14	String replace(CharSequence old, CharSequence new)	It replaces all occurrences of the specified CharSequence.

length()

Returns the length of a string (the number of characters in the string).

```
String str = “VITAP";
System.out.println(str.length()); // Outputs 5
```

charAt(int index)

Returns the character at the specified index.

```
String str = “VITAP";
System.out.println(str.charAt(1)); // Outputs ‘I'
```

substring(int beginIndex, int endIndex)

Returns a substring from the specified beginIndex to endIndex-1.

```
String str = "Hello";
System.out.println(str.substring(1, 3)); // Outputs "el“
```

concat(String str)

Concatenates the specified string to the end of the calling string.

```
String str1 = "Hello";
String str2 = " World";
System.out.println(str1.concat(str2)); // Outputs "Hello World"
```

equals(Object anotherObject) :

Compares the calling string to the specified object. The result is true if and only if the argument is not null and is a String object that represents the same sequence of characters as this object.

```
String str1 = "Hello";
String str2 = "Hello";
System.out.println(str1.equals(str2)); // Outputs true
```

equalsIgnoreCase(String anotherString):

Compares the calling string to another string, ignoring case considerations.

```
String str1 = "hello";
String str2 = "HELLO";
System.out.println(str1.equalsIgnoreCase(str2)); // Outputs true
```

toLowerCase() and toUpperCase()

Converts all the characters in the string to lower case or upper case.

```
String str = "Hello World";
System.out.println(str.toLowerCase()); // Outputs "hello world"
System.out.println(str.toUpperCase()); // Outputs "HELLO WORLD“
```

trim()

Returns a copy of the string, with leading and trailing whitespace omitted.

```
String str = "  Hello World  ";
System.out.println(str.trim()); // Outputs "Hello World"
```

replace(char oldChar, char newChar)

Returns a new string resulting from replacing all occurrences of oldChar in this string with newChar.

```
String str = "Hello World";
System.out.println(str.replace('l', 'p')); // Outputs "Heppo Worpdp"
```